

第18回 JavaScript の基本 3

実習時のフォルダ構成

第 17 回と同じ形式で、Web_名前 の中に 18 フォルダを作る。18 の中に sample と exercise を作り、次のファイルを用意する。

```
Web_名前
├── 18
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、すべて sample/main.js に書く）
- exercise：演習で作るファイル（演習 1～4、発展）

sample/index.html と exercise/index.html には、どちらも次の内容を書く（第 17 回の sample/index.html をコピーして、<title> だけ変えてもよい）。main.js を読み込んで、ブラウザで実行できる最小限の HTML である。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>JavaScriptの基本3</title>
</head>
<body>
  <script src="main.js"></script>
</body>
</html>
```

講義内容

以降の講義内容のコードは、sample/main.js に、上から順に書き足していく。書いたら sample/index.html をブラウザで開き、F12 キーの開発者ツール（コンソール）で結果を確認する。

1. 前回の復習

- 比較演算子（> < >= <= === !==）で 2 つの値を比べると、結果は真偽値（true か false）になる。

```
console.log(30 > 0);           // true
console.log(30 <= 0);          // false
```

- 今回は、この `true` / `false` を使って、「条件によって、実行する処理を変える」方法を学ぶ。

2. if 文

「もし～なら、この処理をする」を書くには、`if` 文を使う。

```
if (条件) {  
  条件が true のときに実行する処理  
}
```

```
const playerHp = 30;  
if (playerHp > 0) {  
  console.log("まだ戦える");  
}  
console.log("ターン終了");
```

- `( )` の中の条件が `true` のときだけ、`{ }` の中が実行される。`false` のときは、`{ }` の中を飛ばして次へ進む。
- `{ }` の外にある `console.log("ターン終了");` は、条件に関係なく実行される。
- `playerHp` を `0` に書き換えて保存し、ブラウザを再読み込みすると、「まだ戦える」が表示されなくなる（確認したら `30` に戻す）。
- この授業では、`{ }` の中は、スペース 2 つ分の字下げ（インデント）をして書く。
- `if ( )` の後ろには `;` を付けない。

`{ }` の中で、変数の値を書き換えることもできる。ゲームで、キャラクターが画面の端から出ないようにする例。

```
const screenWidth = 400;  
let playerX = 390;  
playerX += 20;           // 右へ 20 動く  
if (playerX > screenWidth) {  
  playerX = screenWidth; // 画面の端で止める  
}  
console.log(playerX);    // 400
```

- `playerX` は、キャラクターの横の位置（x 座標）を表す。
- 動いた結果が画面の幅を超えたら、画面の端の値に戻す。

3. if ... else

条件が `false` のときの処理も書きたいときは、`else` を使う。

```
if (条件) {  
  条件が true のときに実行する処理  
} else {  
  条件が false のときに実行する処理  
}
```

```
const enemyHp = 0;
if (enemyHp <= 0) {
  console.log("敵を倒した!");
} else {
  console.log(`敵の残りHP: ${enemyHp}`);
}
```

- 2つの `{ }` のうち、どちらか一方だけが実行される。
- `enemyHp` を `50` に変えると、「敵の残りHP: 50」と表示される。

4. else if

条件を3つ以上に分けたいときは、`else if` を使う。スコアでランクを決める例。

```
const score = 750;
if (score >= 1000) {
  console.log("ランク: S");
} else if (score >= 500) {
  console.log("ランク: A"); // 750 は、ここ
} else {
  console.log("ランク: B");
}
```

- 上から順に条件を調べ、最初に `true` になった `{ }` だけが実行される。残りは調べない。
- どの条件も `false` のときは、最後の `else` が実行される。
- 条件を書く順番に注意する。`score >= 500` を先に書くと、`1200` も「ランク: A」になってしまう。

5. 論理演算子

複数の条件を組み合わせるときは、論理演算子を使う。

記号	読み方	結果が <code>true</code> になるとき
<code>&&</code>	かつ (AND)	左右の両方が <code>true</code> のとき
<code> </code>	または (OR)	左右のどちらかが <code>true</code> のとき
<code>!</code>	ではない (NOT)	<code>true</code> と <code>false</code> を反対にする

```
const enemyX = 250;
console.log(enemyX >= 0 && enemyX <= screenWidth); // true (0 以上 かつ 400 以下)
console.log(enemyX < 0 || enemyX > screenWidth); // false (画面の外ではない)
```

- 「0 以上 400 以下」は、`0 <= enemyX <= 400` のようには書けない（意図どおりに動かない）。この授業では、`&&` で2つの条件をつなげて書く。
- `||` は、キーボードの `Shift + ¥` キーで入力する（日本語キーボードの場合）。

`if` 文の条件にも使える。

```
const hasKey = true;
const level = 8;
if (hasKey && level >= 10) {
  console.log("扉が開いた");
} else {
  console.log("扉は開かない"); // level が 10 未満なので、こちら
}

const isGameOver = false;
console.log(!isGameOver); // true
if (!isGameOver) {
  console.log("ゲームを続ける");
}
```

- `hasKey && level >= 10` : 鍵を持っていて、**かつ**レベルが 10 以上のときだけ `true`。
- `!isGameOver` : 「ゲームオーバーではない」という意味。`isGameOver` が `false` なので、`!isGameOver` は `true` になる。

6. 乱数

ゲームでは、サイコロの目や、敵の出現位置などを、毎回ちがう値にしたいことがある。このような、でたらめな値を「乱数」という。

```
console.log(Math.random()); // 0 以上 1 未満の小数（毎回変わる）
console.log(Math.floor(3.7)); // 3（小数点以下を切り捨てる）
```

- `Math.random()` : 0 以上 1 未満の小数を、ランダムに 1 つ作る。
- `Math.floor()` : `()` の中の数の、小数点以下を切り捨てる。
- どちらも、JavaScript に最初から用意されている機能。`console.log()` と同じように、`()` を付けて使う。`Math` の `M` は大文字。
- ブラウザを再読み込み (F5) するたびに、`Math.random()` の結果が変わる。

2 つを組み合わせると、「1~6 の整数」のような乱数を作れる。

```
const dice = Math.floor(Math.random() * 6) + 1;
console.log(dice); // 1~6 のどれか
```

手順	式	値の範囲
1	<code>Math.random()</code>	0 以上 1 未満の小数
2	<code>Math.random() * 6</code>	0 以上 6 未満の小数
3	<code>Math.floor(Math.random() * 6)</code>	0~5 の整数
4	<code>Math.floor(Math.random() * 6) + 1</code>	1~6 の整数

- この授業では、「`Math.floor(Math.random() * 個数) + 最小値`」の形で使う。
- 例 : 1~6 → 個数 6、最小値 1。10~20 → 個数 11、最小値 10。

敵の出現位置（x 座標）を、画面の幅の中でランダムに決める例。

```
const spawnX = Math.floor(Math.random() * screenWidth);
console.log(spawnX);           // 0~399 のどれか（敵の出現位置）
```

if 文と組み合わせると、「○% の確率で起きること」を作れる。

```
if (Math.random() < 0.3) {
  console.log("会心の一撃!"); // 約 30% の確率
} else {
  console.log("通常の攻撃");
}
```

- `Math.random()` が 0.3 未満になるのは、約 30% の確率。再読み込みを何回かして、結果が変わることを確認する。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて、F12 の開発者ツール（コンソール）で確認する。

演習 1 HP の判定（if ... else）

- 自分の HP（例：50）を `let` で、敵の攻撃力（例：20）を `const` で用意する。
- 自分の HP から、敵の攻撃力を引く。
- HP が 0 以下なら「ゲームオーバー」、そうでなければ「残りHP：○○」と `console.log` で表示する（○○はテンプレートリテラルで表示する）。
- HP の最初の値を 10 に変えて、「ゲームオーバー」と表示されることを確認する。

演習 2 ランクの判定（else if）

- スコアを `const` の変数に入れる（例：650）。
- スコアに応じて、次のようにランクを表示する。
- 1000 以上：「ランク：S」
- 500 以上：「ランク：A」
- 200 以上：「ランク：B」
- それ以外：「ランク：C」
- スコアを 1200、650、300、100 に変えて、それぞれ正しいランクが表示されることを確認する。

演習 3 画面の中にいるか（論理演算子）

- 画面の幅（640）と高さ（480）を `const` で用意する。
- 敵の位置（x 座標と y 座標）を `const` で用意する（例：x は 320、y は 500）。
- 次の 4 つの条件がすべて `true` なら「画面の中」、そうでなければ「画面の外」と表示する。
- x が 0 以上／x が画面の幅以下／y が 0 以上／y が画面の高さ以下

- 敵の位置を変えて、表示が変わることを確認する。

演習4 サイコロ（乱数）

- 1～6 の乱数を作って変数に入れ、「サイコロの目：○」と表示する。
- 目が6 なら「大当たり！」、そうでなければ「はずれ」と表示する。
- 再読み込み（F5）を何回かして、結果が変わることを確認する。

発展（任意）

• 攻撃の命中判定

- ボスの HP（例：30）を `let` で用意する。
- ダメージを 10～20 の乱数で決める。
- 80% の確率で攻撃が命中し、ボスの HP からダメージを引いて「○○のダメージ！」と表示する。命中しなかったときは「ミス！」と表示する。
- そのあと、ボスの HP が 0 以下なら「ボスを倒した！」、そうでなければ「ボスの残りHP：○○」と表示する。

• 自機を画面の端で止める

- 自機の幅（32）、移動の速さ（30）を `const` で、自機の x 座標（例：590）を `let` で用意する。画面の幅は演習3 の値を使う。
- x 座標に速さを足して、右へ動かす。
- 「x 座標 + 自機の幅」が画面の幅より大きくなったら、自機の右端が画面の右端にそろう位置に x 座標を戻し、x 座標を表示する。
- 余裕があれば、x 座標を 10 にしてから速さを引いて左へ動かし、x 座標が 0 未満になったら 0 に戻す処理も書く。